Avaliação de Performance e Consumo Energético de C# em Windows e Linux: Um Estudo Comparativo com FannkuchRedux e BinaryTrees

Gustavo Castro
Universidade do Minho, Departamento de Informática
Mestrado em Engenharia Informática

Maio de 2026

Índice

1 Resumo **3**

2 Introdução **3**

3 Metodologia **4**
3.1 Algoritmos de Benchmark . . . 4
3.1.1 FannkuchRedux (CPU-bound) . . . 4
3.1.2 BinaryTrees (GC-bound / Memory-bound) . . . 4
3.2 Hardware e Software . . . 4
3.3 Medição de Performance — BenchmarkDotNet . . . 5
3.4 Medição de Energia — Intel RAPL . . . 5
3.5 Configuração do Ambiente de Medição . . . 6
3.6 Protocolo de Recolha de Dados . . . 6
3.7 Análise Estatística . . . 6

4 Resultados **7**
4.1 FannkuchRedux (CPU-bound) . . . 7
4.1.1 Visualização dos Dados . . . 7
4.1.2 Comparação de Performance (Tempo) . . . 9
4.1.3 Comparação de Consumo Energético . . . 9
4.1.4 Significância Estatística . . . 9
4.1.5 Correlação Tempo–Energia . . . 10
4.1.6 Variância e Estabilidade . . . 10
4.2 BinaryTrees (GC-bound / Memory-bound) . . . 10
4.2.1 Visualização dos Dados . . . 10
4.2.2 Comparação de Performance (Tempo) . . . 11
4.2.3 Comparação de Consumo Energético . . . 12
4.2.4 Significância Estatística . . . 12

4.2.5 Correlação Tempo–Energia 12
4.3 Análise do Domínio DRAM 12
4.4 Comparação Global entre Algoritmos 13

5 Discussão **14**
5.1 Windows é mais rápido — porquê? 14
5.2 Linux consome menos energia — porquê? 14
5.3 Trade-off velocidade vs. energia 15
5.4 BinaryTrees vs. FannkuchRedux — impacto do perfil de memória 15

6 Ameaças à Validade **15**
6.1 Validade Interna 15
6.2 Validade Externa 16
6.3 Validade de Construção 16

7 Conclusão **16**

1 Resumo

Este trabalho compara a performance e o consumo energético de C# em Windows 11 e Ubuntu 24.04 utilizando dois *benchmarks* complementares: FannkuchRedux, CPU-bound, e BinaryTrees, GC-bound / memory-bound. A medição de energia foi realizada diretamente via Intel RAPL (*Running Average Power Limit*) nos domínios *Package* e DRAM, a medição de tempo via BenchmarkDotNet, e ambos os sistemas operativos correram no mesmo hardware (*dual-boot*), com turbo desativado para garantir comparabilidade.

Em FannkuchRedux, Windows é 43.4% mais rápido para N=11 e 37.7% mais rápido para N=12, mas Linux consome 35% menos energia para N=11 (17.4 J vs. 27.0 J) e 17% menos para N=12 (232.2 J vs. 281.4 J). Em BinaryTrees, Windows é 23% mais rápido para N=16 (148 ms vs. 192 ms) e 11% mais rápido para N=18 (1108 ms vs. 1246 ms), enquanto Linux consome 51% menos energia para N=16 (2.1 J vs. 4.3 J) e 56% menos para N=18 (12.9 J vs. 29.1 J).

A medição independente do domínio DRAM confirma o perfil de cada *workload*: em FannkuchRedux a DRAM representa apenas 2.7% (Linux) / 1.3% (Windows) da energia do *Package*, ao passo que em BinaryTrees sobe para 10.6% (Linux) / 5.0% (Windows), uma razão 4–8× superior que confirma experimentalmente a natureza GC-bound do segundo *benchmark*. Todas as diferenças relevantes são estatisticamente significativas (Mann-Whitney U, $p < 0.001$), confirmadas por testes de permutação independentes. Conclui-se que existe um *trade-off* claro entre velocidade e eficiência energética entre os dois sistemas operativos, com magnitude dependente do perfil de memória do *workload*, com implicações práticas para decisões de *deployment* em ambientes onde o custo energético é relevante.

2 Introdução

A eficiência energética do software é uma preocupação crescente, especialmente em contextos de computação de alto desempenho e *edge computing*. A linguagem C# e o ecossistema .NET são amplamente usados em ambientes de produção que correm tanto em Windows como em Linux. Compreender como o sistema operativo influencia a performance e o consumo energético de um mesmo programa é relevante para decisões de *deployment*.

A maioria dos estudos de performance em .NET foca-se em métricas de tempo de execução, ignorando o consumo energético. Estudos como o de Pereira et al. [5] (*greensoftwarelab/Energy Languages* [2]) demonstram que a linguagem e o ambiente de execução têm impacto significativo na energia consumida. Este trabalho estende essa linha de investigação para comparar dois sistemas operativos com o mesmo hardware e *runtime*.

Para evitar que as conclusões dependam de um único perfil de *workload*, o estudo combina dois *benchmarks* complementares. FannkuchRedux fornece o caso CPU-bound puro (alocação de *heap* desprezável, contenda dominada por aritmética e acesso a *arrays*). BinaryTrees foi adicionado como contraponto GC-bound: a alocação de *heap* é de aproximadamente 227 MB (N=16) a 1037 MB (N=18) por iteração, exercitando intensamente o coletor de lixo e o subsistema de memória. Esta combinação permite distinguir efeitos do *scheduler* e do JIT (visíveis sobretudo em FannkuchRedux) de efeitos do GC e da pressão sobre o subsistema DRAM (visíveis sobretudo em BinaryTrees). Para validar experimentalmente este perfil de memória, o subsistema DRAM é medido de forma independente via Intel RAPL, em adição ao domínio *Package*.

Este trabalho tem como objetivos: (1) medir e comparar o tempo de execução de

C# em Windows 11 e Ubuntu 24.04; (2) medir e comparar o consumo energético via Intel RAPL em ambos os sistemas operativos; (3) avaliar a significância estatística das diferenças observadas; (4) comparar *workloads* CPU-bound (FannkuchRedux) com *workloads* GC-bound (BinaryTrees); (5) medir o subsistema DRAM via Intel RAPL para caracterizar experimentalmente o perfil de memória de cada *benchmark*.

O restante documento está organizado da seguinte forma: a Secção 3 descreve a metodologia e o ambiente experimental; a Secção 4 apresenta os resultados obtidos; a Secção 5 discute as implicações dos resultados; a Secção 6 identifica ameaças à validade; a Secção 7 conclui o trabalho.

3 Metodologia

3.1 Algoritmos de Benchmark

3.1.1 FannkuchRedux (CPU-bound)

O FannkuchRedux é um algoritmo CPU-bound puro, sem I/O, baseado em permutações de arrays, proveniente do *Computer Language Benchmarks Game* (CLBG) [6]. A implementação em C# utilizada é a referência de *greensoftwarelab/Energy-Languages* [2], contribuída por Isaac Gouy, transliterada do programa Java de Oleg Mazurov, com melhorias de concorrência por Peperud [5].

Foram testadas duas variantes: N=11 e N=12 (tamanho do problema, que determina a complexidade fatorial). O algoritmo utiliza *threading* via `Task.Run` com `Environment.ProcessorCount + 1` *workers*. A correção foi verificada: `Compute(7)` devolve *checksum* 228 e *maxFlips* 16, correspondendo ao valor de referência do CLBG. A alocação de *heap* é de aproximadamente 52 KB por iteração (verificado via `MemoryDiagnoser` do BenchmarkDotNet), pelo que o GC é irrelevante para os resultados.

3.1.2 BinaryTrees (GC-bound / Memory-bound)

O BinaryTrees é um algoritmo GC-bound proveniente do CLBG [6] que aloca e percorre árvores binárias recursivas, exercitando intensamente o coletor de lixo e o subsistema de memória. A implementação em C# utilizada é a referência de *greensoftwarelab/Energy-Languages* [2], contribuída por Marek Safar, com melhorias de concorrência por Peperud.

Foram testadas duas variantes: N=16 e N=18. A alocação de *heap* é de aproximadamente 227 MB para N=16 e 1037 MB para N=18 por iteração, ordens de magnitude superior à de FannkuchRedux, o que torna o subsistema de memória o componente dominante do consumo energético. O algoritmo utiliza *threading* via `Task.Run` com *work-stealing* e paralelismo adaptativo (sobre-paralelização para profundidades elevadas). A correção foi verificada: `Compute(10)` devolve *stretchCheck* = 4095 e *longLivedCheck* = 2047, correspondendo ao valor de referência do CLBG.

3.2 Hardware e Software

A Tabela 1 resume as especificações do ambiente de teste.

Tabela 1: Especificações de hardware e software do ambiente de teste.

Campo	**Valor**
Máquina	HP EliteBook 1050 G1
CPU	Intel Core i7-8850H (Coffee Lake, 6C/12T, 2.6 GHz base)
RAM	16 GiB
Armazenamento	Samsung NVMe 512 GB
SO Linux	Ubuntu 24.04.3 LTS, kernel 6.17.0-22-generic
SO Windows	Windows 11 Pro (build 26200.8246)
.NET SDK	9.0.312 (Linux) / 9.0.313 (Windows)
Runtime .NET	9.0.14 (Linux) / 9.0.15 (Windows)
BenchmarkDotNet	0.15.8

3.3 Medição de Performance — BenchmarkDotNet

A ferramenta BenchmarkDotNet 0.15.8 [1] foi usada para medir o tempo de execução. A configuração incluiu `SimpleJob(RuntimeMoniker.Net90)`, `MemoryDiagnoser` e `CsvMeasurementsExpor` O aquecimento (*warm-up*) é realizado automaticamente antes das iterações medidas, com deteção e remoção automática de *outliers* estatísticos pelo BenchmarkDotNet. A compilação foi feita em modo *Release*, x64, com `Optimize=true`. Em Windows foi usado `InProcessEmitToolchain` (necessário para a integração com a LibreHardwareMonitor-Lib).

3.4 Medição de Energia — Intel RAPL

A energia foi medida via Intel RAPL [3] em três domínios: *Package* (`MSR_PKG_ENERGY_STATUS`, 0x611), que inclui todos os núcleos e o *uncore*; PP0/*Cores* (`MSR_PP0_ENERGY_STATUS`, 0x639); e DRAM (`MSR_DRAM_ENERGY_STATUS`, 0x619), que isola o consumo do subsistema de memória. A inclusão do domínio DRAM permite caracterizar experimentalmente o perfil de memória de cada *benchmark* (ver Secção 4.3).

Linux: leitura via sysfs (*kernel powercap*), com os caminhos:

- Package: `/sys/devices/virtual/powercap/intel-rapl/intel-rapl:0/energy_uj`
- PP0/Cores: `/sys/devices/virtual/powercap/intel-rapl/intel-rapl:0/intel-rapl:0:0/e`
- DRAM: `/sys/devices/virtual/powercap/intel-rapl/intel-rapl:0/intel-rapl:0:2/energ`

Windows: leitura via driver WinRing0 (LibreHardwareMonitorLib 0.9.3) [4], com acesso direto ao MSR por reflexão sobre `LibreHardwareMonitor.Hardware.Ring0.ReadMsr` para os três domínios (`0x611`, `0x639`, `0x619`).

A unidade de energia é lida do `MSR_RAPL_POWER_UNIT` (0x606), bits [12:8] — para Coffee Lake: 2^{-14} J/unit ≈ 61 µJ/unit, com o mesmo multiplicador a aplicar-se a todos os três domínios. O método de medição consiste na leitura dos contadores antes e depois de cada iteração; o delta é a energia consumida. O *wraparound* dos contadores de 32 bits foi tratado (período de *wrap* muito superior à duração das *runs*). O CSV de saída inclui as colunas `pkg_energy_uj`, `pp0_energy_uj` e `dram_energy_uj`, além de *timestamp*, duração (ms) e temperatura.

3.5 Configuração do Ambiente de Medição

Para garantir comparabilidade entre os dois sistemas operativos, as seguintes configurações foram aplicadas:

Ambos os SO:

- Turbo Boost desativado (Linux: `/sys/devices/system/cpu/intel_pstate/no_turbo=1`; Windows: `PROCTHROTTLEMAX=99%`)
- Máquina ligada à corrente (AC power)
- Sem outras aplicações intensivas em CPU durante as medições

Linux:

- CPU governor: `performance` (via cpupower)
- Script de lançamento: `scripts/run-linux.sh`

Windows:

- Plano de energia: Desempenho Máximo (GUID `202e5b95...`)
- Processo executado como Administrador (necessário para acesso MSR via WinRing0)

3.6 Protocolo de Recolha de Dados

O número de iterações foi determinado automaticamente pelo BenchmarkDotNet (mínimo de estabilidade estatística). As amostras válidas, após remoção de *outliers* (critério IQR 1.5×), foram:

- FannkuchRedux: Linux N=11: 25 amostras; Linux N=12: 24 amostras; Windows N=11: 23 amostras; Windows N=12: 19 amostras
- BinaryTrees: Linux N=16: 80 amostras; Linux N=18: 22 amostras; Windows N=16: 123 amostras; Windows N=18: 49 amostras

Os dados foram exportados para CSV com *timestamp*, energia (µJ) nos domínios Package/PP0/DRAM, duração (ms) e temperatura.

3.7 Análise Estatística

Foram utilizados os seguintes testes e medidas:

- **Mann-Whitney U** (amostras independentes, não-paramétrico)
- **A/B test por permutação** (10 000 permutações)
- **Correlação de Spearman** ρ (relação não-linear entre tempo e energia)
- **Intervalos de confiança** a 95%, calculados via distribuição t de Student

O nível de significância foi $\alpha = 0.05$. As ferramentas utilizadas foram Python 3, pandas, scipy, seaborn e matplotlib.

4 Resultados

A Tabela 2 apresenta as estatísticas descritivas completas para os oito grupos experimentais (dois algoritmos × dois valores de N × dois SO).

Tabela 2: Estatísticas descritivas por algoritmo, SO e valor de N.

Algoritmo	SO	N	n	Média (ms)	DP (ms)	CV (%)	Pkg (J)	DRAM (J)	DRAM/
FannkuchRedux	Linux	11	25	643.0	11.20	1.74	17.424	0.484	
FannkuchRedux	Linux	12	24	8663.9	27.68	0.32	232.162	6.221	
FannkuchRedux	Windows	11	23	448.3	3.79	0.84	27.013	0.319	
FannkuchRedux	Windows	12	19	6293.9	38.92	0.62	281.436	3.724	
BinaryTrees	Linux	16	80	192.4	8.60	4.47	2.097	0.212	
BinaryTrees	Linux	18	22	1246.1	20.89	1.68	12.878	1.425	
BinaryTrees	Windows	16	123	148.3	8.87	5.98	4.305	0.214	
BinaryTrees	Windows	18	49	1107.8	30.54	2.76	29.088	1.463	

4.1 FannkuchRedux (CPU-bound)

4.1.1 Visualização dos Dados

A Figura 1 mostra a distribuição do tempo de execução por SO e N para FannkuchRedux.

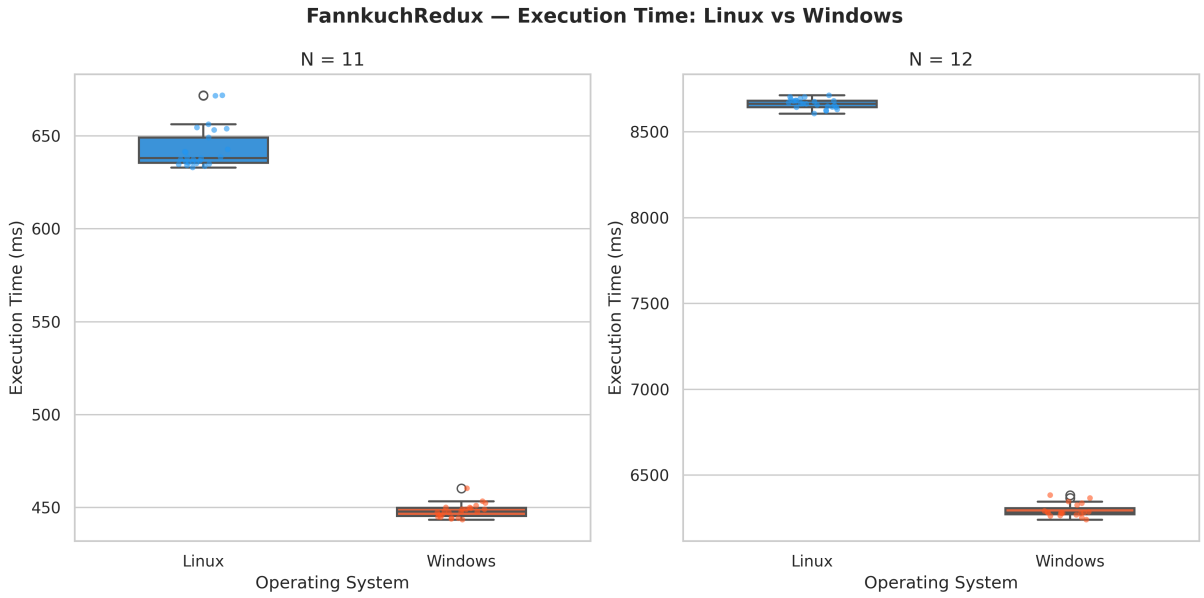

Figura 1: FannkuchRedux: distribuição do tempo de execução para N=11 e N=12 em Linux e Windows. Os pontos representam iterações individuais.

A Figura 2 apresenta a distribuição do consumo energético (domínio *Package*) por SO e N.

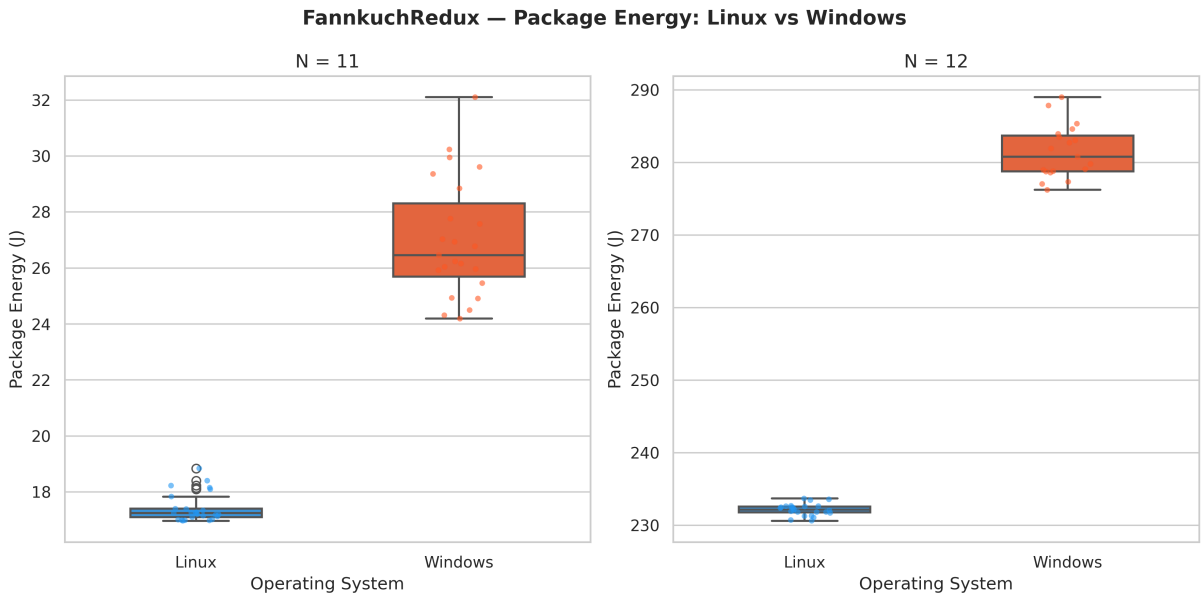

Figura 2: FannkuchRedux: distribuição do consumo energético (domínio *Package*) para N=11 e N=12.

A Figura 3 apresenta as médias e intervalos de confiança a 95% para tempo de execução e energia.

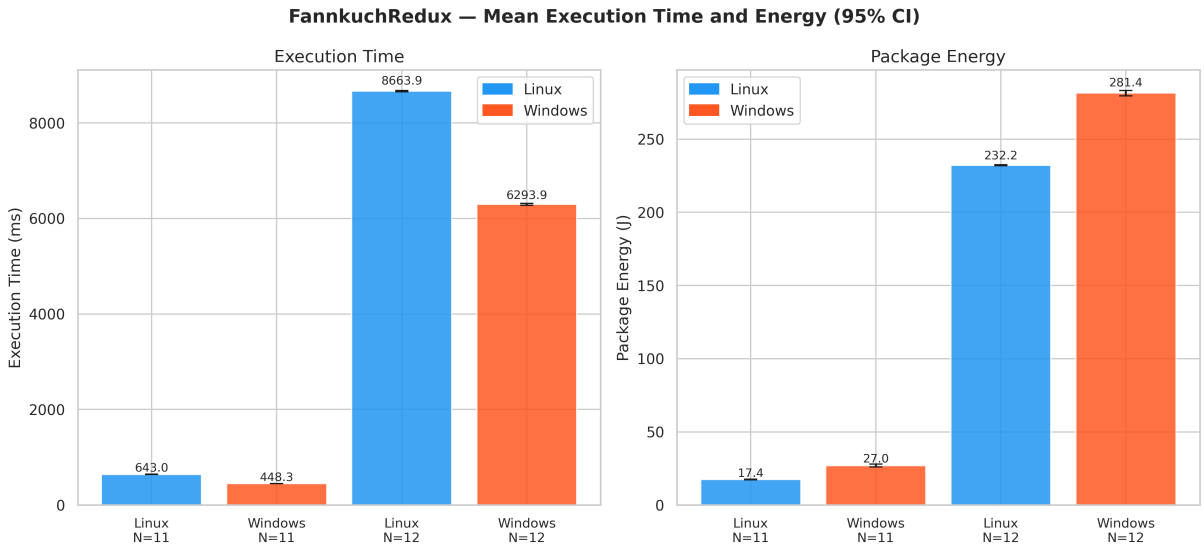

Figura 3: FannkuchRedux: média e IC 95% para tempo de execução (esq.) e energia (dir.) por SO e N.

A Figura 4 mostra a distribuição do tempo de execução por SO e N (histograma com KDE sobreposto).

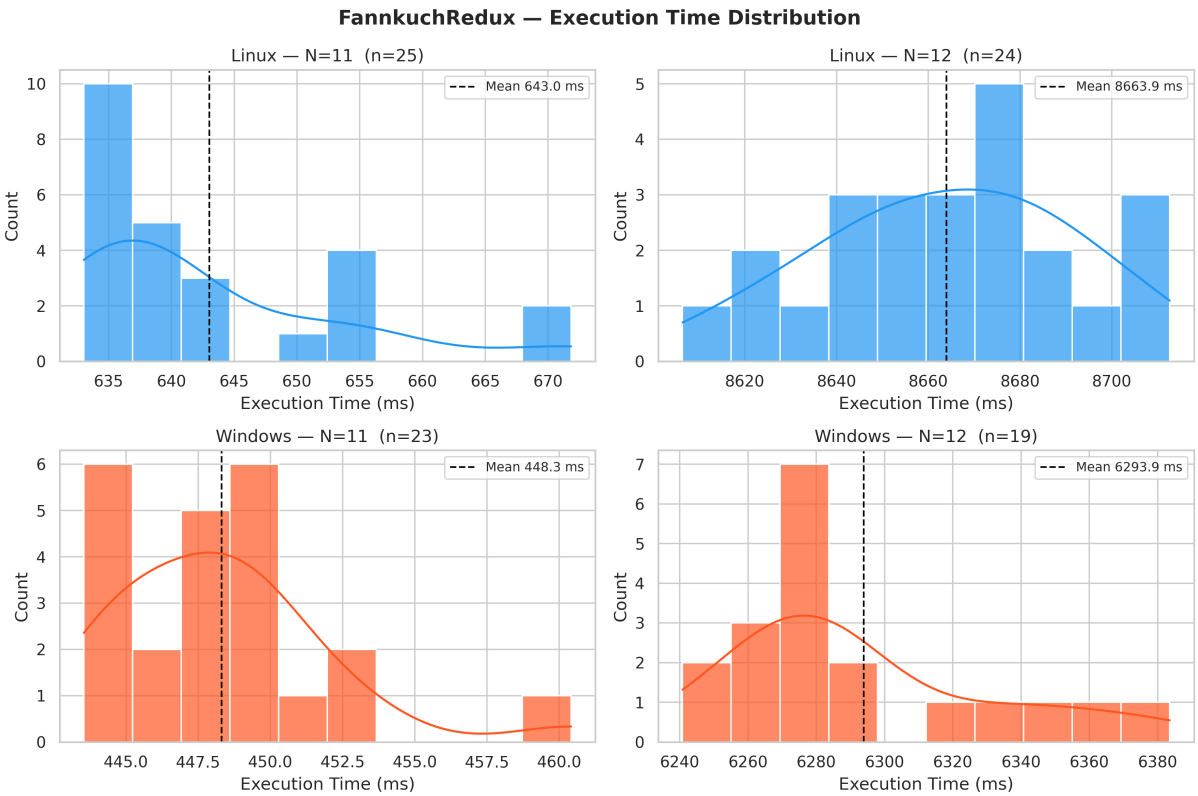

Figura 4: FannkuchRedux: distribuição do tempo de execução por SO e N (histograma + KDE).

4.1.2 Comparação de Performance (Tempo)

Para N=11, Windows executa em 448.3 ms (IC 95%: [446.7, 449.9] ms) vs. Linux 643.0 ms (IC 95%: [638.4, 647.6] ms). **Windows é 43.4% mais rápido.** Os intervalos de confiança não se sobrepõem.

Para N=12, Windows executa em 6293.9 ms (IC 95%: [6275.1, 6312.6] ms) vs. Linux 8663.9 ms (IC 95%: [8652.2, 8675.6] ms). **Windows é 37.7% mais rápido.**

4.1.3 Comparação de Consumo Energético

Para N=11, Linux consome 17.424 J (IC 95%: [17.211, 17.638] J) vs. Windows 27.013 J (IC 95%: [26.095, 27.931] J). **Linux consome 35.5% menos energia.**

Para N=12, Linux consome 232.162 J (IC 95%: [231.820, 232.504] J) vs. Windows 281.436 J (IC 95%: [279.680, 283.193] J). **Linux consome 17.5% menos energia.**

4.1.4 Significância Estatística

A Tabela 3 apresenta os resultados dos testes de hipótese para FannkuchRedux. Todas as diferenças são estatisticamente significativas com $p < 0.001$ ($\alpha = 0.05$).

Tabela 3: FannkuchRedux: resultados dos testes de hipótese para tempo e energia.

N	Métrica	Teste	Estatística	p-value	Significativo
11	Tempo	Mann-Whitney U	575.0	3.16e-9	Sim (***)
11	Energia	Mann-Whitney U	0.0	1.58e-9	Sim (***)
11	Tempo	Permutação A/B	194.7 ms	<0.001	Sim (***)
11	Energia	Permutação A/B	−9.59 J	<0.001	Sim (***)
12	Tempo	Mann-Whitney U	456.0	2.64e-8	Sim (***)
12	Energia	Mann-Whitney U	0.0	1.32e-8	Sim (***)
12	Tempo	Permutação A/B	2370.0 ms	<0.001	Sim (***)
12	Energia	Permutação A/B	−49.27 J	<0.001	Sim (***)

4.1.5 Correlação Tempo–Energia

A correlação de Spearman entre tempo de execução e energia consumida é forte e estatisticamente significativa em ambos os grupos:

- Spearman ρ (FannkuchRedux completo) $= 0.596$, $p < 0.001$
- Spearman ρ (Linux) $= 0.933$, $p < 0.001$
- Spearman ρ (Windows) $= 0.818$, $p < 0.001$

A correlação por SO é muito forte ($\rho > 0.8$ em ambos), o esperado para um *workload* CPU-bound puro. A correlação combinada é mais baixa porque agrega dois SO com regimes energia/tempo distintos.

4.1.6 Variância e Estabilidade

O coeficiente de variação (CV) do Linux mantém-se baixo (0.32%–1.74%), indicando ambiente de medição muito estável. O CV do Windows N=11 (0.84%) é ligeiramente inferior ao CV do Linux N=11 (1.74%) neste *dataset*, ao passo que para N=12 ambos os SO descem para CV inferior a 1%, sugerindo que *workloads* mais longas estabilizam o comportamento de medição.

4.2 BinaryTrees (GC-bound / Memory-bound)

4.2.1 Visualização dos Dados

A Figura 5 mostra a distribuição do tempo de execução de BinaryTrees por SO e N.

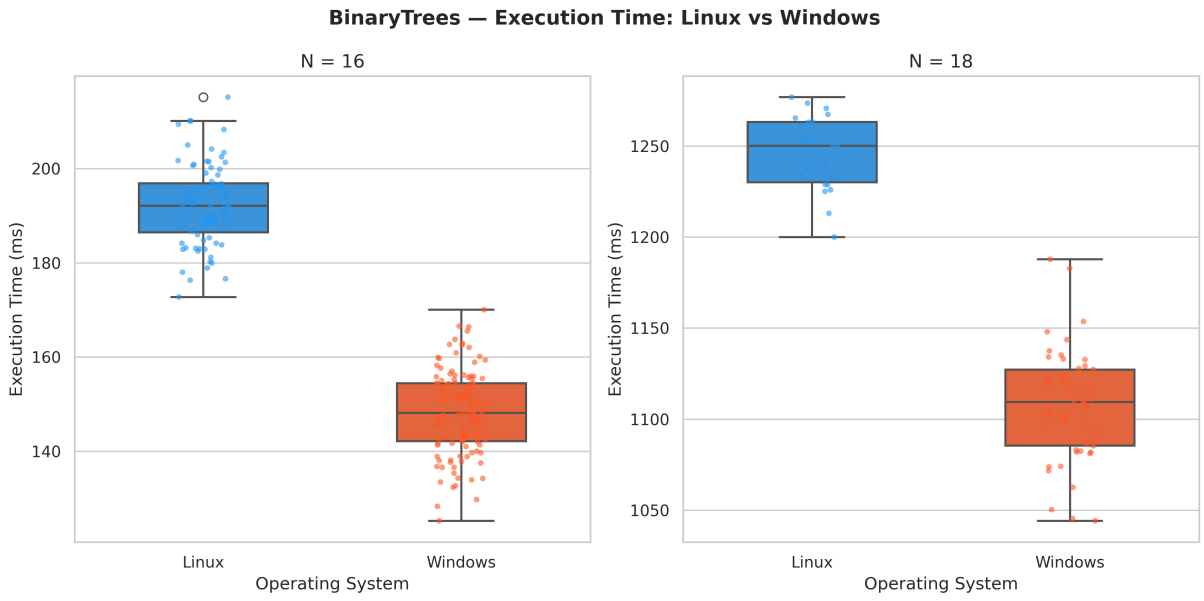

Figura 5: BinaryTrees: distribuição do tempo de execução para N=16 e N=18 em Linux e Windows.

A Figura 6 apresenta a distribuição do consumo energético (domínio *Package*) por SO e N.

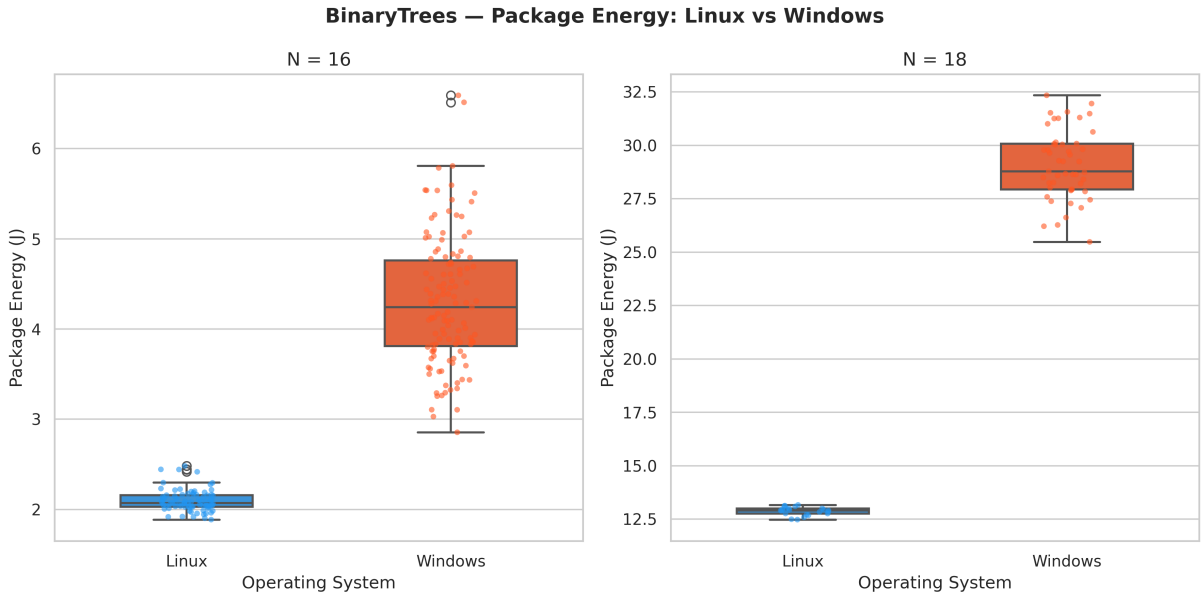

Figura 6: BinaryTrees: distribuição do consumo energético (domínio *Package*) para N=16 e N=18.

4.2.2 Comparação de Performance (Tempo)

Para N=16, Windows executa em 148.3 ms (IC 95%: [146.7, 149.9] ms) vs. Linux 192.4 ms (IC 95%: [190.5, 194.3] ms). **Windows é 22.8% mais rápido** ($p < 0.001$).

Para N=18, Windows executa em 1107.8 ms (IC 95%: [1099.0, 1116.6] ms) vs. Linux 1246.1 ms (IC 95%: [1236.9, 1255.4] ms). **Windows é 11.1% mais rápido** ($p < 0.001$).

4.2.3 Comparação de Consumo Energético

Para N=16, Linux consome 2.097 J (IC 95%: [2.070, 2.123] J) vs. Windows 4.305 J (IC 95%: [4.176, 4.434] J). **Linux consome 51.3% menos energia** ($p < 0.001$).

Para N=18, Linux consome 12.878 J (IC 95%: [12.792, 12.965] J) vs. Windows 29.088 J (IC 95%: [28.624, 29.553] J). **Linux consome 55.7% menos energia** ($p < 0.001$).

4.2.4 Significância Estatística

A Tabela 4 resume os testes de hipótese para BinaryTrees. As diferenças no domínio *Package* (tempo e energia) são todas estatisticamente significativas com $p < 0.001$. A diferença no domínio DRAM é significativa para N=18 mas não para N=16, o que é consistente com o facto de a energia DRAM por iteração ser de apenas 0.21 J em N=16 (próxima do limite de quantização do contador).

Tabela 4: BinaryTrees: resultados dos testes de hipótese para tempo e energia.

N	Métrica	Teste	Estatística	p-value	Significativo
16	Tempo	Mann-Whitney U	9840.0	<0.001	Sim (***)
16	Energia Pkg	Mann-Whitney U	0.0	<0.001	Sim (***)
16	Energia DRAM	Mann-Whitney U	4741.0	0.663	Não
16	Tempo	Permutação A/B	44.1 ms	<0.001	Sim (***)
16	Energia Pkg	Permutação A/B	−2.21 J	<0.001	Sim (***)
18	Tempo	Mann-Whitney U	1078.0	<0.001	Sim (***)
18	Energia Pkg	Mann-Whitney U	0.0	<0.001	Sim (***)
18	Energia DRAM	Mann-Whitney U	298.0	0.003	Sim (**)
18	Tempo	Permutação A/B	138.4 ms	<0.001	Sim (***)
18	Energia Pkg	Permutação A/B	−16.21 J	<0.001	Sim (***)

4.2.5 Correlação Tempo–Energia

A correlação de Spearman entre tempo e energia em BinaryTrees é mais fraca no *dataset* combinado mas continua forte por SO, o que reflete o efeito do GC sobre a relação tempo/energia:

- Spearman ρ (BinaryTrees completo) = 0.318, $p < 0.001$
- Spearman ρ (Linux) = 0.842, $p < 0.001$
- Spearman ρ (Windows) = 0.721, $p < 0.001$

4.3 Análise do Domínio DRAM

A Figura 7 compara o rácio DRAM/*Package* entre os dois *benchmarks* e os dois SO.

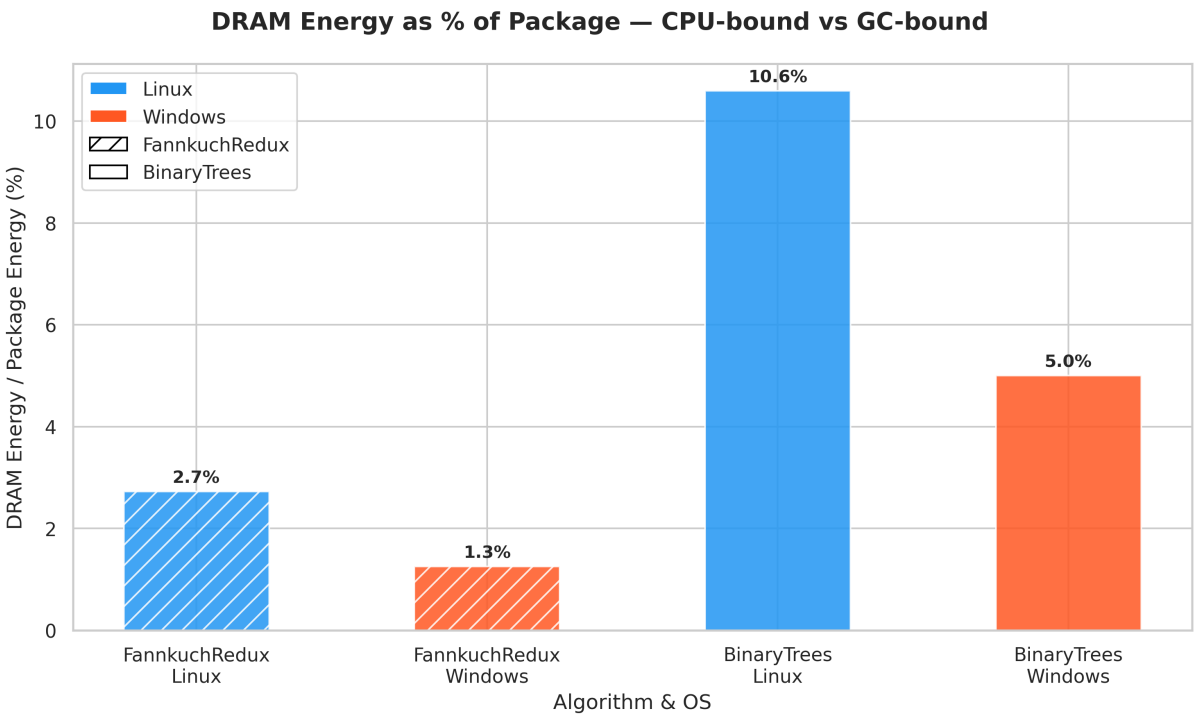

Figura 7: Rácio DRAM/*Package* (%) por algoritmo e SO. BinaryTrees apresenta um rácio 4–8× superior ao FannkuchRedux, confirmando o seu perfil GC-bound / memory-bound.

A medição independente do domínio DRAM produz uma assinatura clara para cada *workload*:

- **FannkuchRedux:** DRAM representa 2.7% da energia do *Package* em Linux e 1.3% em Windows.
- **BinaryTrees:** DRAM representa 10.6% da energia do *Package* em Linux e 5.0% em Windows.

A diferença de 4–8× entre os dois algoritmos confirma experimentalmente que BinaryTrees pressiona o subsistema de memória de forma substancialmente superior, validando a sua classificação como *benchmark* GC-bound / memory-bound. A diferença de rácio entre SO (Linux apresenta sempre rácio mais elevado) é consistente em ambos os algoritmos e sugere que o domínio *Package* no Windows está a contabilizar consumo adicional fora do subsistema DRAM (*e.g.*, frequência/voltagem mais elevada nos núcleos), e não que a DRAM consome menos em Windows — de facto, em valores absolutos, a energia DRAM é praticamente idêntica entre SO (*e.g.*, 0.212 J em Linux vs. 0.214 J em Windows para BinaryTrees N=16).

4.4 Comparação Global entre Algoritmos

A Figura 8 sintetiza, num único gráfico, a comparação entre os dois algoritmos e os dois SO ao nível de tempo e energia por iteração.

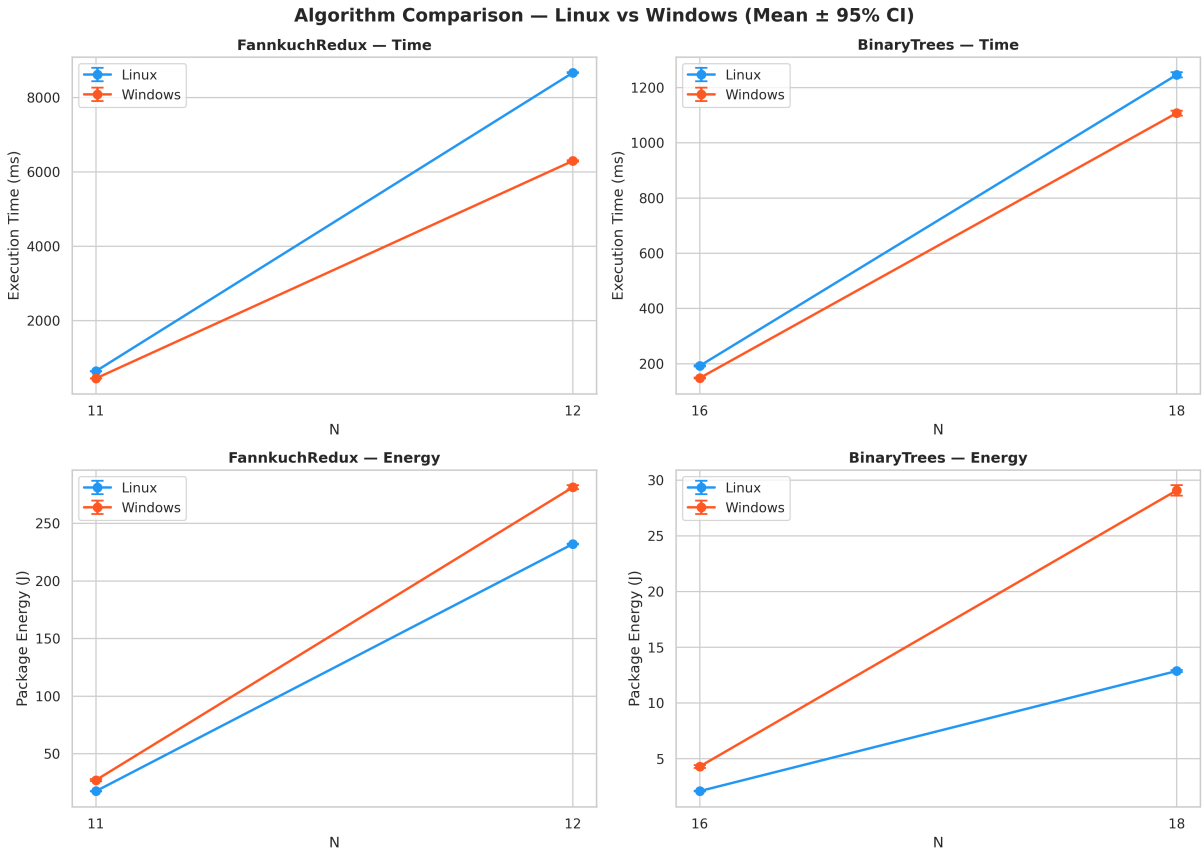

Figura 8: Comparação global tempo vs. energia entre FannkuchRedux e BinaryTrees, por SO e N.

5 Discussão

5.1 Windows é mais rápido — porquê?

Os algoritmos usam `Task.Run` com *workers* dependentes do número de núcleos. O ThreadPool do Windows é historicamente mais agressivo a escalonar *threads* em múltiplos núcleos para *workloads* curtas. O .NET no Windows pode também beneficiar de otimizações específicas da plataforma no JIT (RyuJIT x64). Embora o turbo tenha sido desativado, diferenças residuais de frequência entre os governadores de cada SO não podem ser completamente excluídas. O *scheduler* CFS do Linux (*Completely Fair Scheduler*) prioriza equidade sobre *throughput* máximo, o que pode desfavorecer *workloads* multi-threaded intensivas.

5.2 Linux consome menos energia — porquê?

Windows executa mais rápido, mas consome proporcionalmente mais energia em ambos os *benchmarks*. Este resultado é contra-intuitivo se se assumir que "mais rápido = menos tempo a trabalhar = menos energia". A explicação mais provável é que Windows opera os núcleos a voltagem/frequência mais elevada mesmo com turbo desativado (estados de potência C0 mais agressivos, menor uso de C-states entre tarefas). O governor `performance` do Linux mantém frequência base constante (≈2.6 GHz) com transições suaves de C-state. O plano "Desempenho Máximo" do Windows pode operar em estados de

potência mais elevados de forma mais contínua. Para *workloads* onde o tempo de resposta não é crítico, Linux oferece melhor eficiência energética.

5.3 Trade-off velocidade vs. energia

Existe um *trade-off* claro: Windows oferece aproximadamente 11–43% mais velocidade em troca de 17–56% mais consumo energético, dependendo do algoritmo e de N. A magnitude do efeito varia com a duração da *run*: em FannkuchRedux, para N=12 (*workload* mais longa), a diferença de energia reduz-se em relação a N=11 (17% vs. 35%), sugerindo que o efeito é mais pronunciado em *workloads* curtas onde os *overheads* de *scheduling* dominam. Para aplicações onde a eficiência energética é prioritária (*e.g.*, servidores em *data centers* com custo de energia elevado, dispositivos embebidos), Linux apresenta vantagem clara.

5.4 BinaryTrees vs. FannkuchRedux — impacto do perfil de memória

A comparação cruzada entre os dois *benchmarks* revela um padrão consistente:

- **Vantagem de velocidade do Windows é menor em BinaryTrees** (11–23%) do que em FannkuchRedux (38–43%). A pressão de GC introduz pausas e custos de gestão de memória que parecem equalizar parcialmente a vantagem do *scheduler* do Windows.
- **Vantagem de energia do Linux é maior em BinaryTrees** (51–56%) do que em FannkuchRedux (17–35%). Quando o subsistema de memória se torna dominante, a eficiência energética do Linux ganha ainda mais relevância.
- **O domínio DRAM confirma experimentalmente o perfil de memória distinto**: BinaryTrees apresenta um rácio DRAM/Package 4–8× superior ao de FannkuchRedux (Secção 4.3). Esta evidência por hardware (RAPL DRAM) é independente das estatísticas de *heap* reportadas pelo `MemoryDiagnoser` e valida a classificação dos *benchmarks*.

A leitura conjunta sugere que a pressão de GC *atenua* as diferenças de *scheduler* entre SO mas *amplifica* as diferenças de gestão de energia: a vantagem de velocidade do Windows é menor (a CPU está parcialmente à espera do GC e do subsistema de memória), mas o Windows continua a operar em estados de potência mais elevados durante essas pausas, pelo que o consumo energético cresce desproporcionalmente.

6 Ameaças à Validade

6.1 Validade Interna

Sessões separadas: as medições Linux e Windows foram realizadas em momentos distintos. Fatores como temperatura ambiente e estado do hardware podem introduzir *bias* sistemático.

Temperatura Windows não disponível: impossibilidade de correlacionar temperatura com desempenho no Windows (`cpu_temp_before` = `cpu_temp_after` = -1.0).

Filtro RAPL: a Intel adicionou ruído aleatório às leituras RAPL (IPU 2021.2) como mitigação de ataques *side-channel*. Afeta ambos os SO de forma equivalente; mitigado pelo número elevado de iterações.

6.2 Validade Externa

Hardware único: os resultados foram obtidos num único *laptop* (HP EliteBook 1050 G1, i7-8850H) e podem não generalizar para outros processadores (AMD, ARM) ou fatores de forma (desktop, servidor).

Cobertura de algoritmos: os *benchmarks* cobrem dois perfis distintos (CPU-bound e GC-bound), o que reforça as conclusões em relação à versão anterior do estudo. Resultados podem ainda diferir para *workloads* com I/O intensivo ou execução estritamente *single-threaded*.

Versão .NET: testado com .NET 9. Resultados podem variar com versões anteriores ou futuras.

6.3 Validade de Construção

Dimensão da amostra: as amostras válidas variam por grupo: FannkuchRedux apresenta 19–25 amostras por grupo (limite mínimo do Teorema do Limite Central, mas adequado para os testes não-paramétricos utilizados, válidos para $n \geq 20$); BinaryTrees apresenta 22–123 amostras por grupo, todas acima do limite recomendado.

Oversampling de Windows N=16 em BinaryTrees: o BenchmarkDotNet recolheu 123 amostras válidas para BinaryTrees N=16 em Windows (vs. 80 em Linux para o mesmo grupo) por convergência mais lenta dos critérios de estabilidade estatística da ferramenta. Como as amostras são recolhidas de forma independente e os testes utilizados (Mann-Whitney U, permutação) não exigem grupos equilibrados, o *oversampling* não afeta a validade das comparações — aumenta apenas a precisão da estimativa do grupo correspondente.

Variância em BinaryTrees: o CV de BinaryTrees (1.7%–6.0%) é superior ao de FannkuchRedux (<2%), reflexo do impacto não-determinístico do GC e do subsistema de memória. Os *outliers* foram removidos pelo critério IQR antes dos testes.

7 Conclusão

Este trabalho comparou a performance e o consumo energético de C# (.NET 9) em Windows 11 e Ubuntu 24.04 usando dois *benchmarks* complementares: FannkuchRedux (CPU-bound) e BinaryTrees (GC-bound / memory-bound). A energia foi medida por hardware via Intel RAPL nos domínios *Package* e DRAM, em ambos os SO, no mesmo computador (*dual-boot*).

Os resultados mostram que Windows 11 é consistentemente mais rápido em ambos os *benchmarks* (38–43% em FannkuchRedux, 11–23% em BinaryTrees) mas consome significativamente mais energia (17–35% em FannkuchRedux, 51–56% em BinaryTrees) que Ubuntu 24.04, nas mesmas condições de hardware. A medição independente do domínio DRAM revela que BinaryTrees apresenta um rácio DRAM/*Package* 4–8× superior ao de FannkuchRedux, confirmando experimentalmente o seu perfil GC-bound e validando a escolha do par de *benchmarks*. Todas as diferenças relevantes são estatisticamente

significativas (Mann-Whitney U, $p < 0.001$) e confirmadas por testes de permutação independentes.

Os resultados evidenciam um *trade-off* claro entre velocidade e eficiência energética dependente do SO, com magnitude que varia com o perfil de memória do *workload*: em *workloads* GC-bound, a vantagem de velocidade do Windows reduz-se mas a vantagem energética do Linux amplifica-se. Para aplicações onde o custo energético é prioritário — como servidores em larga escala ou sistemas embebidos — Linux apresenta vantagem significativa.

Como trabalho futuro sugere-se: (1) replicar o estudo noutros tipos de hardware (desktop, servidor, ARM); (2) incluir *workloads single-threaded* e com I/O intensivo; (3) comparar versões de .NET (8 vs. 9 vs. 10); (4) avaliar o impacto do modo `GC Server` vs. `GC Workstation` em *workloads* GC-bound como BinaryTrees; (5) investigar o impacto de diferentes configurações do *scheduler* Linux (CFS vs. EEVDF, *tickless kernel*).

Referências

[1] BenchmarkDotNet contributors. BenchmarkDotNet. GitHub, 2024. URL https://github.com/dotnet/BenchmarkDotNet.

[2] greensoftwarelab. Energy-languages. GitHub, 2017. URL https://github.com/greensoftwarelab/Energy-Languages.

[3] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3B: System Programming Guide, Part 2*, 2023. Chapter 14: RAPL — Running Average Power Limit.

[4] LibreHardwareMonitor contributors. LibreHardwareMonitor. GitHub, 2024. URL https://github.com/LibreHardwareMonitor/LibreHardwareMonitor.

[5] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, SLE 2017, pages 256–267. ACM, 2017. doi: 10.1145/3136014.3136031.

[6] The Computer Language Benchmarks Game contributors. Computer language benchmarks game. https://benchmarksgame-team.pages.debian.net/benchmarksgame/, 2024.